

Project_M

스터디 보고서

2025 / 03 / 19

박정하

목차

1. 코딩테스트 [미지의 공간 탈출]

2. 코딩테스트 [고대 문명 유적 탐사]

1. 코딩테스트[메두사와 전사들]

메두사와 전사들

- 코드 구상

1. 입력값을 각각 적절한 변수에 저장
2. 메두사의 시선
3. 메두사 공원까지 최단거리(BFS)
4. 병사 -> 메두사 이동(맨해튼 거리)
5. While문에서 time을 1씩 증가시키며, 각 턴마다 순서대로 과정 진행

메두사와 전사들

■ 코드 동작 과정

1. 메두사의 BFS(좌표 추출)

✓ 더 이상 지도에서 벽(1)을 신경 쓸 필요 없음.

```
0 0 0 0
0 0 0 M
0 0 1 1
1 0 0 P
메두사의 BFS 결과 : [(1, 3), (1, 2), (1, 1), (2, 1), (3, 1), (3, 2), (3, 3)]
```

2. 지도를 모두 0으로 초기화 시키고, 전사의 위치에 1을 넣음

```
0 0 0 1
1 0 M 0
0 0 1 0
0 1 0 0
```

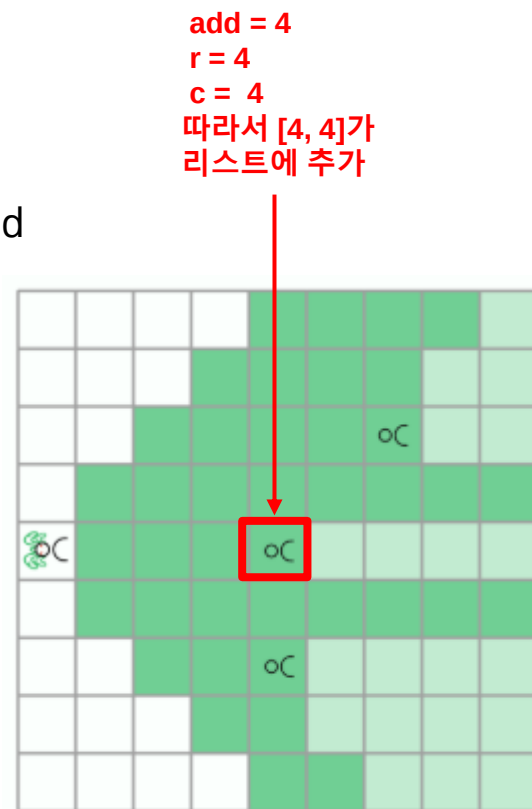
3. While 루프 문 안에서 time을 1씩 증가시키며

메두사의 이동 → 메두사의 시선 → 전사 이동 1 → 전사이동 2 을 진행

메두사와 전사들

- Scan(사용할 지도, 메두사의 위치, 방향)
 - 현재 메두사가 있는 줄 탐색
 - Add를 반복문 안에서 add를 1씩 증가시킴.
 - Add는 탐색할 열의 위치에 더해짐
 - 0이 아닌 것이 탐지되면 반복문 종료, 탐색 위치 리스트에 append

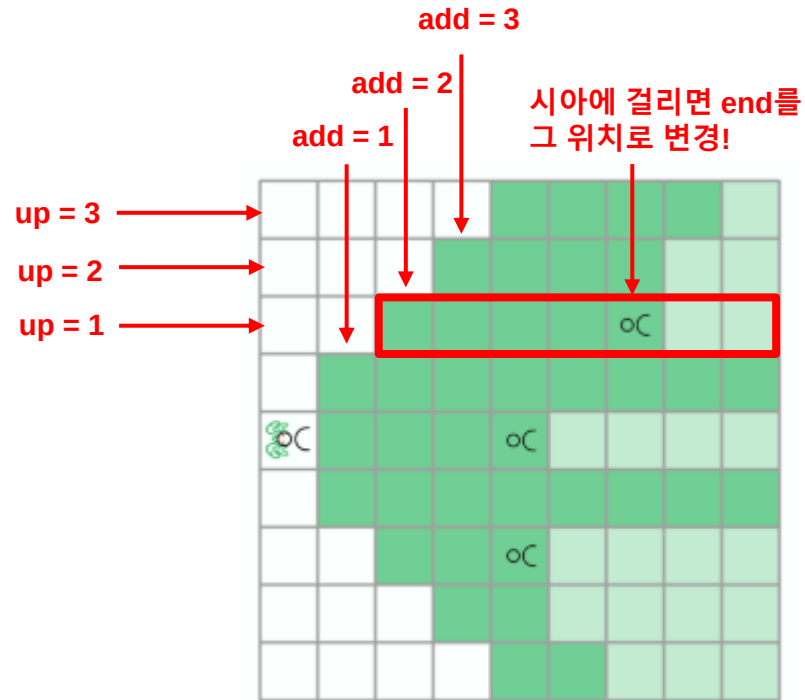
```
def scan(tar_map, now_pos, dir): # (사용하는 지도, 메두사의 위치, 방향)
    pos_dic = []
    if dir == "R": # 오른쪽 방향 탐색
        add = 1
        while now_pos[1]+add < N:
            r, c = now_pos[0], now_pos[1]+add
            if tar_map[r][c] != 0 :
                pos_dic.append([r, c])
                break
            add += 1
```



메두사와 전사들

- Scan(사용할 지도, 메두사의 위치, 방향)
 - 그 위의 줄 탐색 while문 안에서 0보다 클 때(맵 범위 안)까지 모두 탐색

```
up = 1
add = 1
end = N-1
define_end = 0
while now_pos[0]-up >=0: #위
    r = now_pos[0] - up
    start = now_pos[1]+add
    to_end = 0
    if define_end > 1:
        define_end -= 1
    if define_end == 1:
        if end!=N-1:
            end +=1
    while start + to_end <= end and end != N:
        c = start + to_end
        if tar_map[r][c] != 0 :
            end = c
            pos_dic.append([r, c])
            define_end =3
            break
        to_end +=1
    add += 1
    up +=1
```



define_end : end가
두 행 뒤부터
늘어나도록 하게 구현

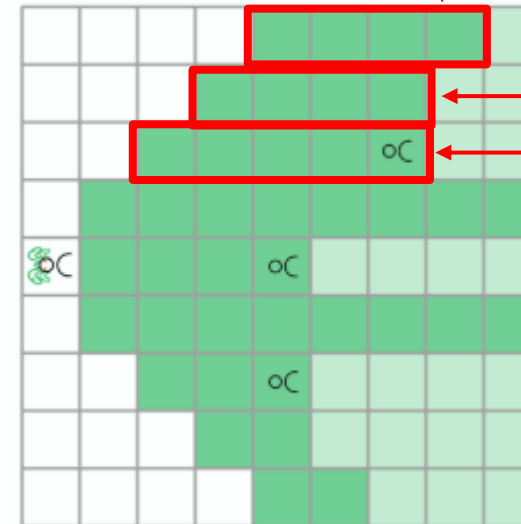
메두사와 전사들

- Scan(사용할 지도, 메두사의 위치, 방향)
 - Define_end : end가 다음 두번째 행부터 늘어나게 하도록 구현

```
up = 1
add = 1
end = N-1
define_end = 0
while now_pos[0]-up >= 0: #위
    r = now_pos[0] - up
    start = now_pos[1]+add
    to_end = 0
    if define_end > 1:
        define_end -= 1
    if define_end == 1:
        if end!=N-1:
            end +=1
    while start + to_end <= end and end != N:
        c = start + to_end
        if tar_map[r][c] != 0 :
            end = c
            pos_dic.append([r, c])
            define_end =3
            break
        to_end +=1
    add += 1
    up +=1
```

0	0	0	1
1	0	M	0
0	0	1	0
0	1	0	0

3. 두번째 행
define_end(2) > 1 이므로 -1,
-1하면 1이므로 end += 1,
End가 7로 늘어난 것을 확인

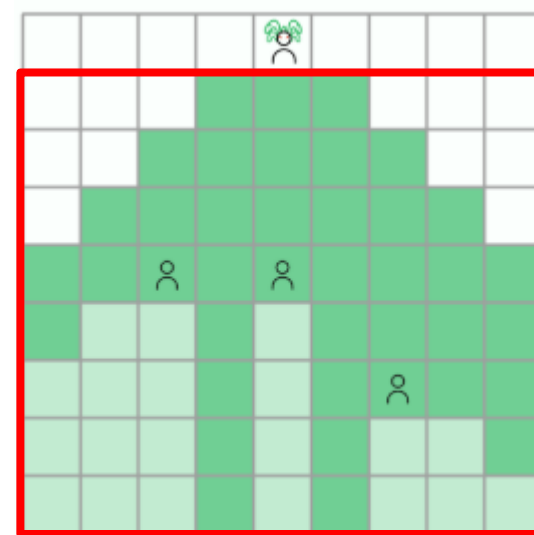
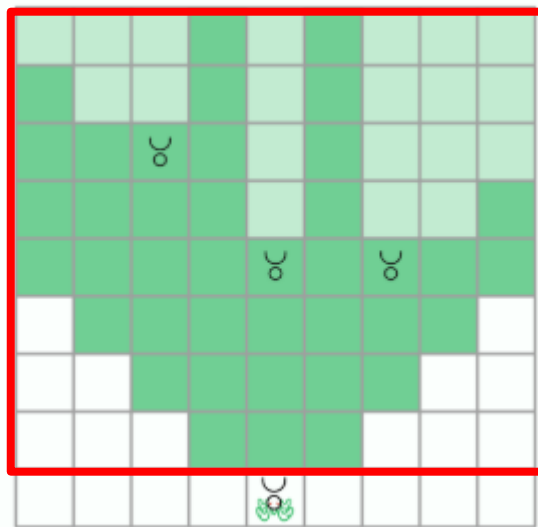
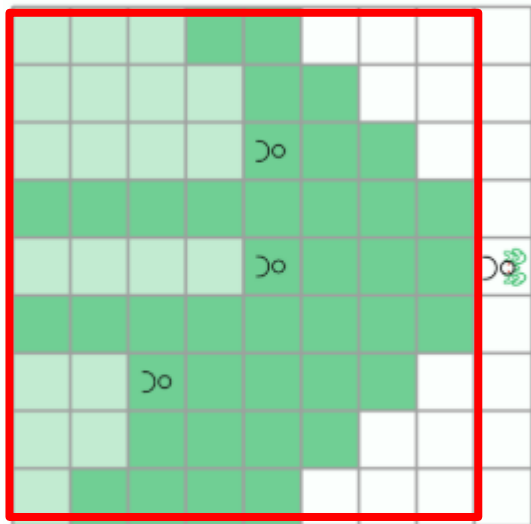
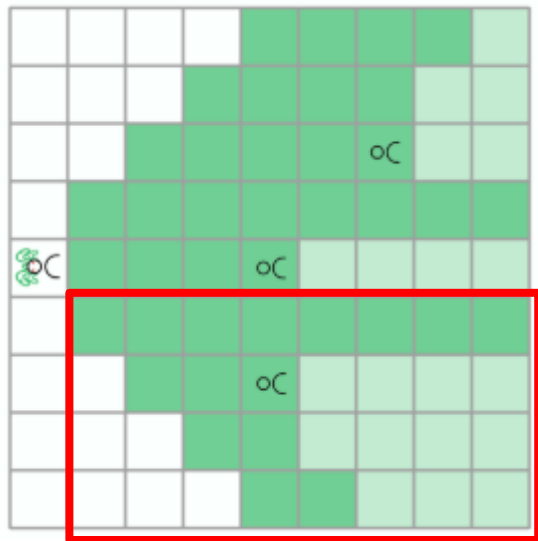


2. 다음 행에서
define_end(3) > 1 이므로
-1, end는 그대로 6

1. 탐색 후 define_end = 3
end = 6

메두사와 전사들

- Scan(사용할 지도, 메두사의 위치, 방향)
 - 동일한 방법으로 아래, 좌, 상 하 방향 모두 구현



메두사와 전사들

- Scan(사용할 지도, 메두사의 위치, 방향)
 - Return은 탐지된 전사의 위치 정보, 그리고 병사의 수

```
return pos_dic, len(pos_dic)
```

메두사와 전사들

- Scan_one_d(대상 지도)

- 네 방향 탐색 모두 진행 후 각 방향마다 탐지된 전사의 위치를 받아옴.
- 전사가 한 칸에 두 명 있는 경우 탐지된 병사 수에 2를 더해줌.

```
def scan_one_d(tar_map):  
    now_pos = find_M(tar_map)  
    Top, _ = scan(tar_map, now_pos, "T")  
    Bottom, _ = scan(tar_map, now_pos, "B")  
    Right, _ = scan(tar_map, now_pos, "R")  
    Left, _ = scan(tar_map, now_pos, "L")  
    n_top, n_bot, n_rig, n_lef = 0, 0, 0, 0  
  
    for r, c in Bottom:  
        n_bot += int(tar_map[r][c])  
    for r, c in Top:  
        n_top += int(tar_map[r][c])  
    for r, c in Right:  
        n_rig += int(tar_map[r][c])  
    for r, c in Left:  
        n_lef += int(tar_map[r][c])
```

메두사와 전사들

- Scan_one_d(대상 지도)

- 방향 설정은 탐지된 전사 수가 같을 경우 상, 하, 좌, 우 의 우선순위를 따름
- 만약 value 안에 값이 여러 개일 경우 ex) [4, 0, 3, 4]
- Max_indices = [0, 3]일 것이고
- 이 중 max_indices[0] = 0 이므로
- 상 방향에 대한 탐지된 전사 위치, 그리고 탐지 방향 return

```
list_list = [Top, Bottom, Left, Right]
values = [n_top, n_bot, n_lef, n_rig]
max_value = max(values)
direction_list = ["T", "B", "L", "R"]
max_indices = [i for i, v in enumerate(values) if v == max_value]
if len(max_indices) != 0: # top bot lef rig
    max_indices = max_indices[0]
return list_list[max_indices], direction_list[max_indices]
```

메두사와 전사들

- Find_M(대상 지도)
 - 대상 지도에서 메두사의 위치를 반환하도록 하는 함수

```
def find_M(tar_map):  
    for c in range(N):  
        for l in range(N):  
            if tar_map[l][c] == "M" :  
                return (l,c)
```

메두사와 전사들

■ Make_BFS_map(대상 지도)

- 길 - 벽 - 공원 - 집 지도를 이용하여 BFS진행
- 최단 거리이며, 이동 우선순위는
상, 하, 좌, 우
- 결과로 출발위치(집의 위치)를 포함한 공원까지의
BFS 경로 return

```
0 0 0 0
0 0 0 M
0 0 1 1
1 0 0 P
메두사의 BFS 결과 : [(1, 3), (1, 2), (1, 1), (2, 1), (3, 1), (3, 2), (3, 3)]
```

```
def make_BFS_map(tarr_map):
    count = 1
    visited = set()
    queue = deque([(home, [home])])
    visited.add(home)
    while queue:
        for _ in range(len(queue)):
            (r, c), path = queue.popleft()

            for r2, c2 in ((r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)):
                if not (0 <= r2 < N and 0 <= c2 < N):
                    continue
                if (r2, c2) in visited:
                    continue
                if tarr_map[r2][c2] == 0:
                    queue.append(((r2, c2), path + [(r2, c2)]))
                    visited.add((r2, c2))
                    tarr_map[r2][c2] = count

                if (r2, c2) == park:
                    return True, path + [(r2, c2)]

            count += 1
    return False, [] # 도달하지 못한 경우
```

메두사와 전사들

- Make_sol_pos_list(대상 지도)

- 대상 지도에서 모든 전사의 위치를 반환

```
def make_sol_pos_list(tar_map):  
    return_sol_list = []  
    for l in range(N):  
        for c in range(N):  
            if tar_map[l][c] != 0 and tar_map[l][c] != "M":  
                return_sol_list.append([l,c])  
    return return_sol_list
```

- Make_move_list(감지된 전사 리스트, 모든 전사 리스트)

- 이동 가능한 전사 위치 반환

```
def make_move_list(ditected_list, sol_pos_list):  
    re = []  
    for i in range(len(sol_pos_list)):  
        if sol_pos_list[i] not in ditected_list:  
            re.append(sol_pos_list[i])  
    return re
```

메두사와 전사들

- Make_clear_map(N)
 - N*N크기의 0으로 채워진 배열 생성

```
def make_clear_map(N):  
    clear_map = [[0 for _ in range(N)] for _ in range(N)]  
    return clear_map
```


메두사와 전사들

- Sol_BFS_Move1(대상 지도, 이동할 전사 위치 정보)
 - 메두사의 위치 [M_r, M_c]
 - 전사의 위치 [S_r, S_c]
 - 맨해튼 거리 기준으로 메두사와 가까워지도록 한 칸 이동
 - 이 때 우선순위는 상, 하, 좌, 우
 - 처음위치, 다음 이동 후 위치를 path에 넣고 return
- Sol_BFS_Move2(대상 지도, 이동할 전사 위치 정보)
 - 1과 동일
 - 최단거리 우선순위가 좌, 우, 상, 하

```
def sol_BFS_Move1(tar_map, move_sol_list):  
    M_tar = find_M(tar_map)  
    M_r, M_c = M_tar[0], M_tar[1]  
    S_r, S_c = move_sol_list[0], move_sol_list[1]  
    path = [(S_r, S_c)]  
    if S_r > M_r: #메두사보다 아래있으면  
        path.append((S_r-1, S_c))  
    elif S_r < M_r: #하하  
        path.append((S_r+1, S_c))  
    if S_c > M_c :#좌  
        path.append((S_r, S_c-1))  
    elif S_c < M_c : #우  
        path.append((S_r, S_c+1))  
    return(path)
```

```
def sol_BFS_Move2(tar_map, move_sol_list):  
    M_tar = find_M(tar_map)  
    M_r, M_c = M_tar[0], M_tar[1]  
    S_r, S_c = move_sol_list[0], move_sol_list[1]  
    pat = [(S_r, S_c)]  
    if S_c > M_c :#좌  
        pat.append((S_r, S_c-1))  
    elif S_c < M_c: #우  
        pat.append((S_r, S_c+1))  
    if S_r > M_r : #상  
        pat.append((S_r-1, S_c))  
    elif S_r < M_r : #하  
        pat.append((S_r+1, S_c))  
    return(pat)
```

메두사와 전사들

- Sol_move(대상 지도, 경로1, 경로2, 방향)



메두사와 전사들

- Sol_move(대상 지도, 경로1, 경로2, 방향)
 - 경로 1, 경로 2 중 이동 가능한 위치를 찾는다.
 - Return값은
전사 수, 원래 위치, 이동 위치

```
def sol_move(tarmap,path,path3,direction):
    location_medusa = find_M(Map)
    ditected_list,_ = scan(tarmap, location_medusa,direction) #이미 탐지된 전전사들

    if [path[1][0],path[1][1]] in ditected_list and [path3[1][0],path3[1][1]] in ditected_list: #이미 탐지된 위치로는 이동 불가
        return 0, [path[0][0],path[0][1]], [path[0][0],path[0][1]]

    else:
        tarmap[path[0][0]][path[0][1]], tarmap[path[1][0]][path[1][1]] = tarmap[path[1][0]][path[1][1]], tarmap[path[0][0]][path[0][1]] #경로 1로 전사 이동동
        ditected_list,_ = scan(tarmap, location_medusa,direction) # 위치 스캔

        if [path[1][0],path[1][1]] in ditected_list: # 경로 1로 이동했을때 스캔되면
            tarmap[path[0][0]][path[0][1]], tarmap[path[1][0]][path[1][1]] = tarmap[path[1][0]][path[1][1]], tarmap[path[0][0]][path[0][1]] #경로 1 이동 취소

            tarmap[path3[0][0]][path3[0][1]], tarmap[path3[1][0]][path3[1][1]] = tarmap[path3[1][0]][path3[1][1]], tarmap[path3[0][0]][path3[0][1]] #경로 2로 이동
            ditected_list,_ = scan(tarmap, location_medusa,direction) #경로 2 위치 스캔

            if [path3[1][0],path3[1][1]] in ditected_list: #경로 2로 이동했을때 스캔되면
                tarmap[path3[0][0]][path3[0][1]], tarmap[path3[1][0]][path3[1][1]] = tarmap[path3[1][0]][path3[1][1]], tarmap[path3[0][0]][path3[0][1]] #경로 2 이동 취소

                return 0, [path[0][0],path[0][1]], [path[0][0],path[0][1]]

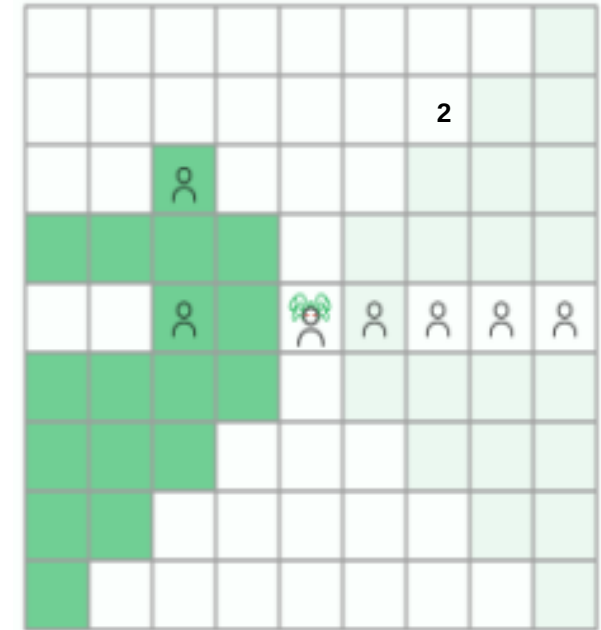
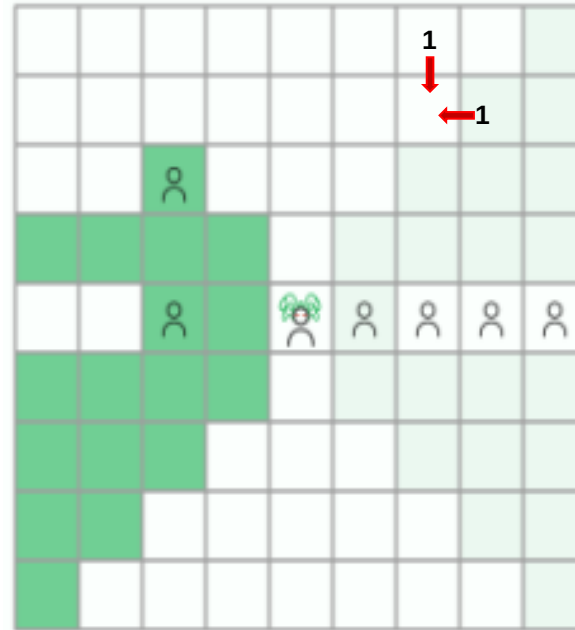
            else: #경로1은 스캔, 경로 2는 이동 가능
                tarmap[path3[0][0]][path3[0][1]], tarmap[path3[1][0]][path3[1][1]] = tarmap[path3[1][0]][path3[1][1]], tarmap[path3[0][0]][path3[0][1]]
                return int(tarmap[path3[0][0]][path3[0][1]]), [path3[1][0],path3[1][1]], [path3[0][0],path3[0][1]]

        else: #경로 1로 이동 가능
            tarmap[path[0][0]][path[0][1]], tarmap[path[1][0]][path[1][1]] = tarmap[path[1][0]][path[1][1]], tarmap[path[0][0]][path[0][1]]
            return int(tarmap[path[0][0]][path[0][1]]), [path[1][0], path[1][1]], [path[0][0], path[0][1]]
```

메두사와 전사들

- Remake_map(대상 지도, 이동할 위치, 탐지된 전사 위치, 이동전 위치)

```
def remake_map(tarmap, sol_list, ditedced_list, brfore_list):  
    num_list = brfore_list + ditedced_list  
    New_Map = [[0 for _ in range(N)] for _ in range(N)]  
    total_list = sol_list + ditedced_list  
    for i in range(len(total_list)):  
        row, col = total_list[i][0], total_list[i][1]  
        num = int(tarmap[num_list[i][0]][num_list[i][1]])  
        New_Map[row][col] += num  
    M_tar = find_M(tarmap)  
    New_Map[M_tar[0]][M_tar[1]] = "M"  
    return New_Map
```



메두사와 전사들

- 전체 코드 흐름

0	0	0	1
1	0	0	M
0	0	1	0
0	1	0	0

초기 지도



0	0	0	1
1	0	M	0
0	0	1	0
0	1	0	0

메두사 이동



0	0	0	0
0	1	M	1
0	0	1	0
0	1	0	0

병사 1차 이동



0	0	0	0
0	0	M	0
0	0	1	0
0	1	0	0

병사 2차 이동

4 2 2

결과

0	0	0	0
0	0	M	0
0	0	1	0
0	1	0	0

초기 지도



0	0	0	0
0	M	0	0
0	0	1	0
0	1	0	0

메두사 이동

0 2 0

결과

2. 코딩테스트[고대 문명 유적 탐사]

고대 문명 유적 탐사

- 코드 구상

1. 입력값을 각각 적절한 변수에 저장
2. 회전 구현
3. 유물 획득 구현
4. 새로 생겨나는 조각 구현

고대 문명 유적 탐사

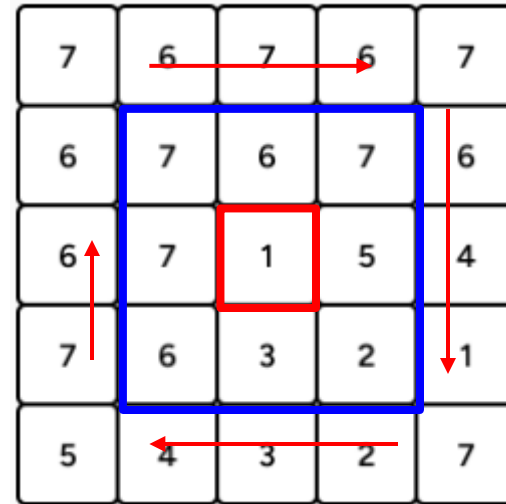
■ Tample_rotate(유적 지도, 타겟, 각도)

```
def tample_rotate(tample_map1, target, degree):
    target = target
    rotate_target = [(target[0]-1, target[1]-1), (target[0]-1, target[1]),
                    (target[0]-1, target[1]+1), (target[0], target[1]+1),
                    (target[0]+1, target[1]+1), (target[0]+1, target[1]),
                    (target[0]+1, target[1]-1), (target[0], target[1]-1)]
    rotate_list = deque()
    for r, c in rotate_target:
        rotate_list.append(tample_map1[r][c])

    if degree == 90:
        rotate_list.rotate(2)
    elif degree == 180:
        rotate_list.rotate(4)
    elif degree == 270:
        rotate_list.rotate(6)

    for i, (r, c) in enumerate(rotate_target):
        tample_map1[r][c] = rotate_list[i]

    return tample_map1
```



90도 -> 2

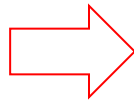
7 6 7 5 2 3 6 7

6 7 7 6 7 5 2 3

고대 문명 유적 탐사

- Search(유적 지도)

7	6	7	6	7
6	6	7	7	6
6	3	1	6	4
7	2	5	7	1
5	4	3	2	7



1	4	3	1	1
4	4	3	3	1
4	1	1	1	1
1	1	1	1	1
1	1	1	1	1

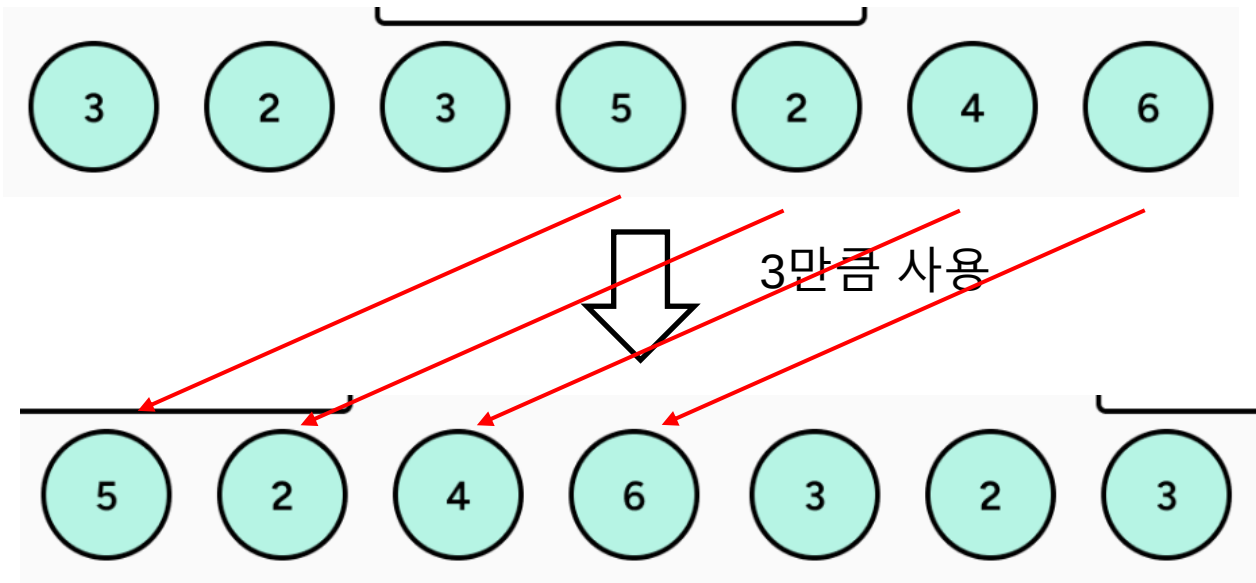
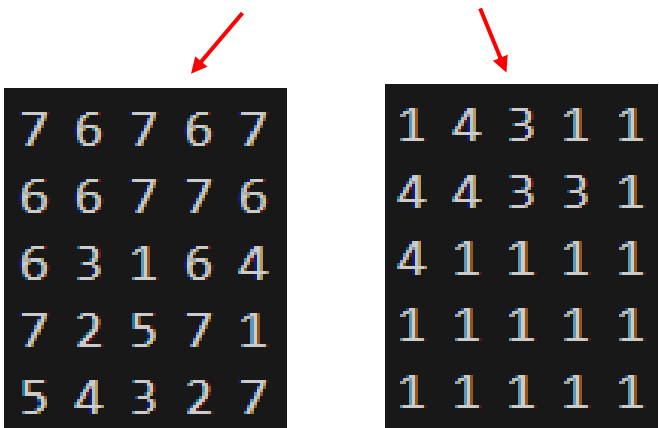
And 7 return

고대 문명 유적 탐사

```
def search(tample_map2):
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    adjacent_counts = [[0] * 5 for _ in range(5)]
    for i1 in range(5):
        for j1 in range(5):
            current_value = tample_map2[i1][j1]
            visited = []
            new_queue = deque()
            visited.append((i1, j1))
            new_queue.append((i1, j1))
            # print(new_queue)
            while new_queue:
                i, j = new_queue.popleft()
                # print('='*20)
                # print(new_queue)
                for dx, dy in directions:
                    nx, ny = i + dx, j + dy
                    if 0 <= nx < 5 and 0 <= ny < 5:
                        if tample_map2[nx][ny] == current_value and (nx,ny) not in visited:
                            new_queue.append((nx,ny))
                            visited.append((nx,ny))
                for i, j in visited:
                    adjacent_counts[i][j] = len(visited)
    count = 0
    for i1 in range(5):
        for j1 in range(5):
            if adjacent_counts[i1][j1] > 2:
                count += 1
    return count, adjacent_counts
```

고대 문명 유적 탐사

- Delet_fill(유적 지도, 숫자 지도, 전에 조각 사용 횟수)



고대 문명 유적 탐사

- Delet_fill(유적 지도, 숫자 지도, 전에 조각 사용 횟수)
 - 우선순위: 열 번호 작은 순, 행 번호 큰 순

1	4	3	1	1
4	4	3	3	1
4	1	1	1	1
1	1	1	1	1
1	1	1	1	1

```
def delete_fill(tarmap,num_map,before):
    fill_list = []
    next_relics_start = 0
    relics.rotate(-before)
    for j in range(5):
        for i in range(4,-1,-1):
            if num_map[i][j] > 2:
                fill_list.append((i,j))
    for index, (i1,j1) in enumerate(fill_list):
        tarmap[i1][j1] = relics[index]
        next_relics_start +=1
    return tarmap , next_relics_start
```